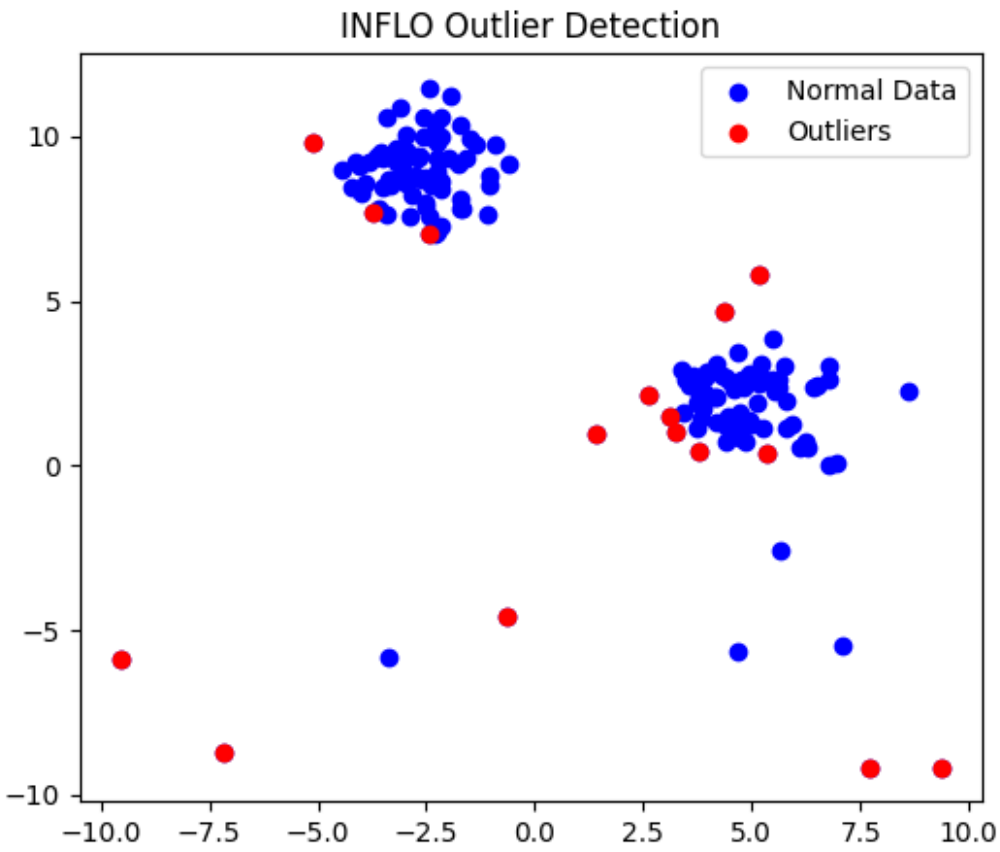
```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from scipy.spatial.distance import cdist

# 1. Generate Dataset
X, _ = make_blobs(n_samples=150, centers=2, cluster_std=1.0,
                  random_state=42)

# Add some artificial outliers
outliers_true = np.random.uniform(low=-10, high=10, size=(10, 2))
X = np.vstack([X, outliers_true])

# 2. Parameters
k = 10

# 3. Distance Matrix
dist_matrix = cdist(X, X)

# 4. KNN (exclude self)
knn_indices = np.argsort(dist_matrix, axis=1)[:, 1:k+1]

# 5. Reverse Nearest Neighbors
rnn_list = [set() for _ in range(len(X))]

for i in range(len(X)):
    for neighbor in knn_indices[i]:
        rnn_list[neighbor].add(i)

# 6. Influenced Neighborhood
influenced_neighbors = []

for i in range(len(X)):
    knn_set = set(knn_indices[i])
    rnn_set = rnn_list[i]
    influenced_neighbors.append(list(knn_set.union(rnn_set)))

# 7. Density Function
def compute_density(i, neighbors):
    if len(neighbors) == 0:
        return 0
    dists = dist_matrix[i, neighbors]
    avg_dist = np.mean(dists)
    return 1 / (avg_dist + 1e-10)  # avoid division by zero

# 8. Compute INFLO
inflo_scores = []

for i in range(len(X)):
    neighbors = influenced_neighbors[i]

    density_p = compute_density(i, neighbors)
```

```
    if density_p == 0:
        inflo_scores.append(0)
        continue

    neighbor_densities = [
        compute_density(j, influenced_neighbors[j])
        for j in neighbors
    ]

    avg_density = np.mean(neighbor_densities)

    inflo = avg_density / density_p
    inflo_scores.append(inflo)

inflo_scores = np.array(inflo_scores)

# 9. Detect Outliers
threshold = np.percentile(inflo_scores, 90)
outlier_mask = inflo_scores > threshold
detected_outliers = X[outlier_mask]

# 10. Visualization
plt.figure(figsize=(6, 5))

plt.scatter(X[:, 0], X[:, 1], c='blue', label='Normal Data')

if len(detected_outliers) > 0:
    plt.scatter(detected_outliers[:, 0],
                detected_outliers[:, 1],
                c='red', label='Outliers')

plt.title("INFLO Outlier Detection")
plt.legend()
plt.show()
```


INFLO Outlier Detection
Normal Data
Outliers
10
5
0
−5
−10
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0